

RAPPORT TECHNIQUE

Projet Technologies IoT

Conception et prototypage d'un système IoT de bout en bout

Système IoT de monitoring et de load balancing intelligent de 4 serveurs

Équipe Projet

MAALEJ Youssef | MEZGHANI Ahmed | SIALA Nasr

GI2 S3

Date de soumission : 29 mars 2026

Table des Matières

Table des Matières	2
1. Introduction	4
2. Identification des Besoins Techniques	5
2.1 Problème résolu	5
2.2 Besoins fonctionnels	5
2.3 Besoins non fonctionnels	5
2.4 Scénarios d'utilisation	6
Scénario 1 — Surcharge soudaine	6
Scénario 2 — Dégradation progressive	6
Scénario 3 — Panne réseau	6
2.5 Grandeurs mesurées	6
2.6 Actionneurs et commandes	6
2.7 Data Analytics envisagée	7
2.8 Contraintes de déploiement	7
3. Architecture Globale du Système	8
3.1 Vue d'ensemble	8
3.2 Description des composants	8
3.2.1 Nœuds IoT (Servers 1 à 4)	8
3.2.2 Broker MQTT — Mosquitto	9
3.2.3 Moteur de règles — Node-RED	9
3.2.4 Base de données — InfluxDB	9
3.2.5 Tableau de bord — Grafana	9
3.3 Stack technologique	9
3.4 Topics MQTT	10
3.5 Topologie réseau	10
4. Conception de la Partie Dispositif	11
4.1 Choix de la simulation	11
4.2 Architecture logicielle des nœuds IoT	11
4.2.1 Bibliothèques utilisées	11
4.2.2 Structure d'un nœud IoT simulé	11
4.2.3 Format du payload MQTT	11
4.2.4 Stratégies d'économie d'énergie (conceptuelle)	11
4.3 Architecture matérielle de référence	12
5. Conception de la Logique Applicative	13
5.1 Approche retenue	13
5.2 Gestion de la connectivité et ingestion des données	13
5.3 Device Management	13
5.4 Gestion des données	14

5.5 Traitement et moteur de règles (Node-RED)	14
5.5.1 Flux principal de traitement	14
5.5.2 Algorithme de load balancing	14
5.6 Analytics et détection d'anomalies	15
5.7 Interface utilisateur — Grafana	15
5.8 Schéma fonctionnel du flux de données	16
6. Aspects Sécurité	17
7. Conclusion	18
8. Références	19

1. Introduction

L'Internet des Objets (IoT) transforme profondément la manière dont les systèmes informatiques sont supervisés et gérés. Dans le contexte des infrastructures serveur, la gestion dynamique des charges de travail représente un défi critique : un déséquilibre de charge entre serveurs entraîne des dégradations de performance, une surconsommation d'énergie et un risque de panne.

Ce projet propose une solution IoT complète de monitoring et de load balancing intelligent de 4 serveurs. Chaque serveur est modélisé comme un nœud IoT publiant ses métriques en temps réel via le protocole MQTT. Une plateforme locale — composée de Mosquitto, Node-RED et InfluxDB — collecte, analyse et réagit automatiquement aux déséquilibres détectés. L'ensemble des données et décisions est visualisé dans un tableau de bord Grafana.

Ce rapport présente la conception complète du système, de l'identification des besoins à l'architecture détaillée, en passant par la conception matérielle (simulée) et la logique applicative.

2. Identification des Besoins Techniques

2.1 Problème résolu

Dans une infrastructure multi-serveurs, il est fréquent que certains serveurs soient surchargés tandis que d'autres restent sous-utilisés. Sans mécanisme de détection et de redistribution automatique, cette situation conduit à :

- Des ralentissements ou indisponibilités sur les serveurs surchargés
- Un gaspillage des ressources sur les serveurs sous-chargés
- Une dégradation de l'expérience utilisateur et des niveaux de service (SLA)
- Une difficulté de supervision manuelle dans un contexte distribué

La solution IoT proposée automatise la détection et la réaction à ces déséquilibres en temps réel.

2.2 Besoins fonctionnels

Le système doit assurer les fonctions suivantes :

- Acquérir en continu les métriques de charge (CPU, RAM, réseau) de chaque serveur
- Transmettre ces métriques via MQTT vers une plateforme centrale
- Stocker l'historique des métriques dans une base de données time-series
- Détecter automatiquement les situations de surcharge (dépassement de seuils)
- Déclencher une action de redistribution de charge vers les serveurs disponibles
- Publier des alertes lorsqu'un serveur dépasse les seuils définis
- Afficher en temps réel l'état de tous les serveurs dans un tableau de bord
- Permettre la consultation de l'historique des charges et des événements

2.3 Besoins non fonctionnels

Les contraintes de qualité attendues sont les suivantes :

- Performance : latence de bout en bout < 2 s entre mesure et action
- Disponibilité : système opérationnel 24h/24, 7j/7
- Scalabilité : architecture extensible à plus de 4 serveurs
- Sécurité : communication MQTT avec authentification par identifiants
- Fiabilité : reconnexion automatique des nœuds IoT en cas de perte réseau
- Autonomie : fonctionnement sans Internet (déploiement local)
- Coût : solution 100 % open-source, sans abonnement cloud

2.4 Scénarios d'utilisation

Scénario 1 — Surcharge soudaine

Le Server 1 reçoit une rafale de requêtes. Son CPU dépasse 80 %. Node-RED détecte l'anomalie et publie une commande de migration sur le topic loadbalancer/commands. Les Servers 3 et 4 (moins chargés) prennent en charge une partie des tâches. Le Server 1 revient sous son seuil en moins de 10 secondes.

Scénario 2 — Dégradation progressive

Le Server 2 présente une fuite mémoire simulée : sa RAM augmente lentement au fil du temps. InfluxDB enregistre la tendance. Grafana génère une alerte visuelle. Un opérateur peut intervenir avant la saturation.

Scénario 3 — Panne réseau

La connexion entre le Server 4 et le broker MQTT est interrompue. Le nœud IoT tente une reconnexion automatique toutes les 5 secondes. Grafana affiche le serveur en état 'Hors ligne'. Node-RED redirige les tâches vers les autres serveurs.

2.5 Grandeurs mesurées

Métrique	Unité	Fréquence	Seuil alerte	Précision attendue
CPU usage	(%)	5 s	> 80 %	± 1 %
RAM usage	(%)	5 s	> 85 %	± 1 %
Network I/O	Mbps	5 s	> 900 Mbps	± 5 Mbps
Load score	[0-100]	5 s	> 80	Calculé

Le score de charge global est calculé comme une moyenne pondérée : $\text{Load} = 0.5 \times \text{CPU} + 0.3 \times \text{RAM} + 0.2 \times \text{Network_ratio}$. Ce score unifié facilite la comparaison entre serveurs et le déclenchement des règles de load balancing.

2.6 Actionneurs et commandes

Le système intègre un actionneur logiciel : le moteur de load balancing de Node-RED. Son rôle est de redistribuer les tâches lorsqu'un déséquilibre est détecté.

- Mode de commande : publication MQTT sur le topic loadbalancer/commands
- Temps de réponse : < 500 ms après détection du dépassement de seuil
- Contrainte : ne pas déclencher la migration si le serveur cible dépasse 70 %

2.7 Data Analytics envisagée

- Visualisation en temps réel : courbes de charge, jauges CPU/RAM sur Grafana
- Génération d'alertes : dépassement de seuils avec horodatage
- Analyse historique : consultation des pics de charge sur les 24 dernières heures
- Détection d'anomalies : règles basées sur des seuils absolus et variations rapides
- Tendances : identification des serveurs chroniquement surchargés

2.8 Contraintes de déploiement

- Environnement : déploiement en réseau local (LAN) ou sur machine unique (loopback)
- Simulation : les 4 nœuds IoT sont des scripts Python — pas de hardware physique
- Disponibilité réseau : fonctionnement sans Internet
- Maintenance : tous les services sont conteneurisés ou installés en local

3. Architecture Globale du Système

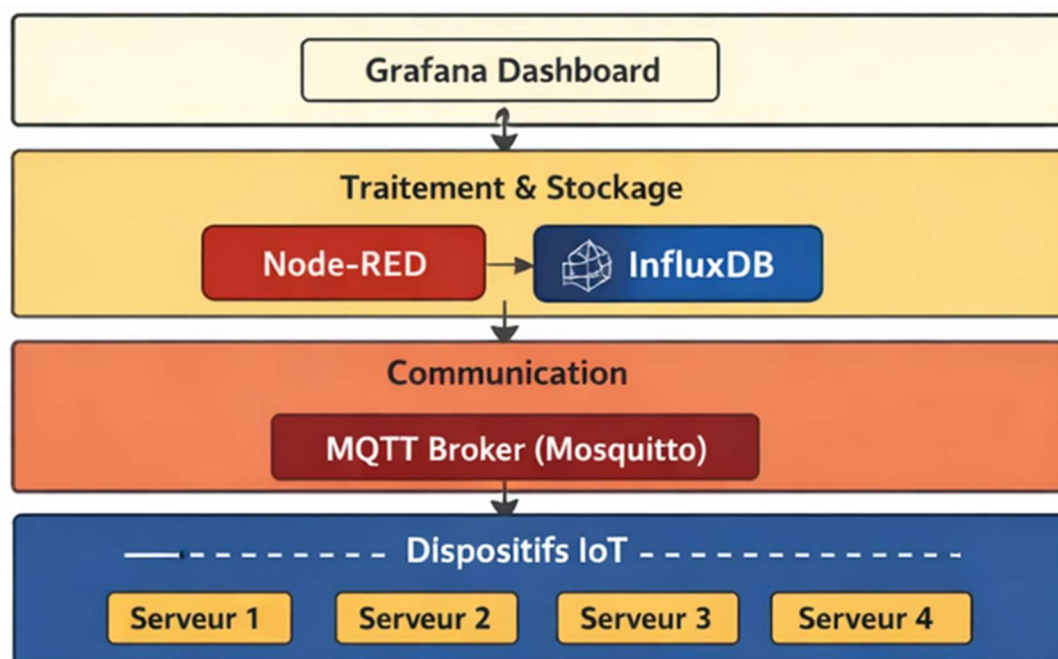
3.1 Vue d'ensemble

L'architecture du système repose sur le modèle en couches standard IoT, adapté au contexte de supervision de serveurs. Elle se compose de cinq couches fonctionnelles :

1. Couche Perception : acquisition des métriques par les nœuds IoT (scripts Python simulant les 4 serveurs)
2. Couche Réseau : transmission des messages via le protocole MQTT, broker Mosquitto
3. Couche Plateforme IoT : traitement des règles (Node-RED), stockage (InfluxDB)
4. Couche Application : visualisation et alertes (Grafana)

Le flux de données est unidirectionnel pour les métriques (nœuds → broker → plateforme → application) et bidirectionnel pour les commandes de load balancing (plateforme → nœuds).

Architecture IoT de bout en bout



3.2 Description des composants

3.2.1 Nœuds IoT (Serveurs 1 à 4)

Chaque serveur est simulé par un script Python exécuté en parallèle. Le script génère des valeurs de CPU, RAM et réseau selon une distribution réaliste incluant des pics aléatoires. Il se connecte au broker Mosquitto et publie ses métriques toutes les 5 secondes sur des topics MQTT dédiés.

3.2.2 Broker MQTT — Mosquitto

Mosquitto est le point central de routage des messages MQTT. Il reçoit les publications des 4 nœuds IoT, les distribue aux abonnés (Node-RED et InfluxDB) et gère les commandes de load balancing publiées par Node-RED vers les nœuds.

3.2.3 Moteur de règles — Node-RED

Node-RED souscrit aux topics MQTT de tous les serveurs. Pour chaque message reçu, il évalue le score de charge et applique les règles : si un serveur dépasse le seuil, Node-RED publie une commande de migration et une alerte. Il gère également un mécanisme anti-oscillation (cooldown de 30 secondes) pour éviter les migrations répétées.

3.2.4 Base de données — InfluxDB

InfluxDB est une base de données spécialisée dans les séries temporelles (time-series). Elle stocke toutes les métriques reçues avec leur horodatage. Elle est directement interrogée par Grafana via son API pour construire les graphiques historiques.

3.2.5 Tableau de bord — Grafana

Grafana fournit l'interface utilisateur principale du système. Il affiche en temps réel : les courbes de charge de chaque serveur, les jauges CPU/RAM/réseau, les alertes actives et l'historique des migrations. Les alertes Grafana peuvent être configurées pour envoyer des notifications par email ou webhook.

3.3 Stack technologique

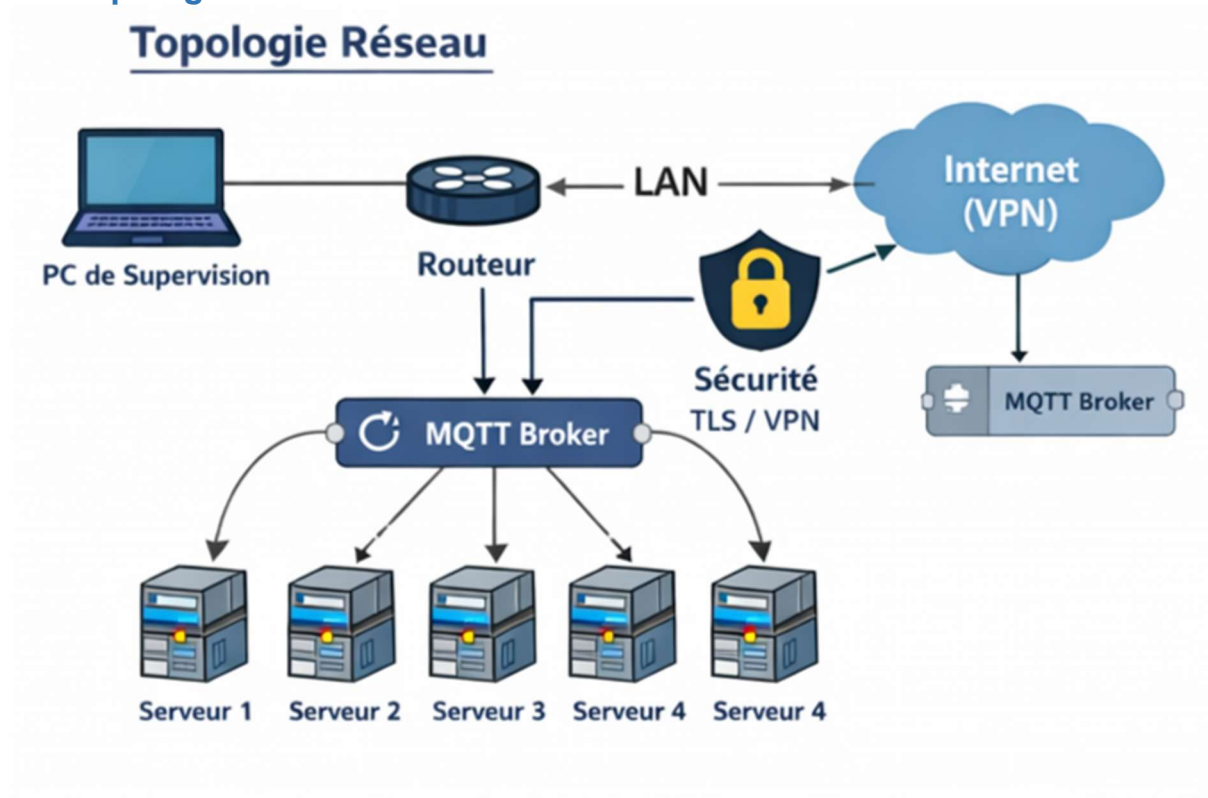
Couche	Composant	Technologie	Rôle
Perception	Nœuds IoT	Python 3 + paho-mqtt	Simuler 4 serveurs, publier métriques
Réseau	Broker MQTT	Mosquitto 2.x	Routage des messages MQTT
Plateforme	Moteur de règles	Node-RED	Détection surcharge, load balancing
Plateforme	Stockage	InfluxDB 2.x	Métriques time-series
Application	Dashboard	Grafana	Visualisation et alertes temps réel

3.4 Topics MQTT

Topic MQTT	Description
servers/server1/cpu	Utilisation CPU du serveur 1 (%)
servers/server1/ram	Utilisation RAM du serveur 1 (%)
servers/server1/network	Bande passante réseau serveur 1 (Mbps)
servers/+cpu	Wildcard — CPU de tous les serveurs
loadbalancer/commands	Commandes de migration de charge
loadbalancer/alerts	Alertes de surcharge publiées par Node-RED

La structure hiérarchique des topics MQTT permet une souscription sélective (par serveur ou par métrique) et facilite l'extension du système à un nombre plus grand de serveurs.

3.5 Topologie réseau



Tous les composants sont déployés sur un réseau local ou sur une machine unique (connexions en loopback 127.0.0.1). La topologie est de type étoile centrée sur le broker Mosquitto. Les ports utilisés sont :

- 1883 (TCP) : communication MQTT non chiffrée (développement/test)
- 3000 : interface web Grafana
- 8086 : API HTTP InfluxDB
- 1880 : interface web Node-RED

4. Conception de la Partie Dispositif

4.1 Choix de la simulation

En l'absence de hardware physique disponible, les nœuds IoT sont simulés par des scripts Python. Ce choix est justifié par :

- La disponibilité immédiate (pas de délai d'approvisionnement matériel)
- La reproductibilité : les scénarios de charge sont parfaitement contrôlables
- La cohérence avec les recommandations du cahier des charges (simulateur d'objet IoT)
- La possibilité de générer des données réalistes incluant des anomalies ciblées

4.2 Architecture logicielle des nœuds IoT

4.2.1 Bibliothèques utilisées

- paho-mqtt : client MQTT Python pour la connexion au broker et la publication
- psutil (optionnel) : lecture des vraies métriques si exécuté sur la machine hôte
- random / math : génération de données simulées réalistes
- threading : publication périodique sans bloquer la boucle principale
- json : sérialisation des données au format JSON pour le payload MQTT

4.2.2 Structure d'un nœud IoT simulé

Chaque nœud IoT est un script Python autonome structuré autour des modules suivants :

- Module de connexion : gestion de la connexion MQTT avec reconnexion automatique
- Module de génération de données : production de métriques CPU/RAM/réseau avec variations réalistes (bruit gaussien + pics aléatoires)
- Module de publication : envoi périodique (toutes les 5 s) des métriques au format JSON
- Module de souscription aux commandes : réception des ordres de load balancing et simulation de la réduction de charge

4.2.3 Format du payload MQTT

Chaque message publié par un nœud IoT contient le payload JSON suivant :

```
{ "server_id": "server1", "timestamp": 1711620000, "cpu": 72.4, "ram": 61.8, "network": 345.2, "load_score": 68.3 }
```

4.2.4 Stratégies d'économie d'énergie (conceptuelle)

Bien que le déploiement soit logiciel, les stratégies suivantes seraient appliquées dans un déploiement hardware réel :

- Mode veille entre deux publications : sleep(5) entre chaque cycle
- Publication conditionnelle : envoi uniquement si la variation dépasse 2 % (delta-based)
- QoS 0 pour les métriques (fire-and-forget) afin de minimiser le trafic réseau
- QoS 1 pour les alertes et commandes afin de garantir la livraison

4.3 Architecture matérielle de référence

Si le projet devait être déployé sur hardware physique, le choix se porterait sur les composants suivants :

- Microcontrôleur : ESP32 (dual-core 240 MHz, Wi-Fi intégré, faible consommation)
- Capteurs : INA219 (courant/tension), DS18B20 (température de la carte), GPIO pour signaux d'état
- Communication : Wi-Fi 802.11 b/g/n vers le broker Mosquitto
- Alimentation : 5V USB ou batterie LiPo 3.7V avec module de charge
- Interfaces : I2C pour les capteurs, UART pour le débogage

5. Conception de la Logique Applicative

5.1 Approche retenue

L'approche retenue est une plateforme IoT personnalisée locale, entièrement basée sur des outils open-source. Cette approche a été préférée à une plateforme cloud commerciale pour les raisons suivantes :

Critère	Plateforme Cloud	Plateforme Locale (choisie)
Coût	Abonnement mensuel	Open-source, gratuit
Latence	Variable (Internet)	< 10 ms (LAN)
Confidentialité	Données chez le fournisseur	Données en local
Disponibilité	Dépend d'Internet	Autonomie sans Internet
Complexité	Faible (services gérés)	Modérée (installation manuelle)

Cette approche garantit une totale maîtrise des données, une latence minimale (essentielle pour le load balancing) et une solution reproductible sans dépendance externe.

5.2 Gestion de la connectivité et ingestion des données

Le broker Mosquitto centralise tous les flux de données. Il est configuré avec :

- Authentification par nom d'utilisateur / mot de passe pour chaque nœud
- ACL (Access Control List) : chaque serveur ne peut publier que sur ses propres topics
- Persistance des sessions : les messages publiés en QoS 1 sont stockés en cas de déconnexion temporaire
- Logging des connexions et déconnexions pour la surveillance du device management

5.3 Device Management

La gestion des dispositifs est assurée par deux mécanismes :

- Last Will and Testament (LWT) : chaque nœud déclare un message de déconnexion automatique publié par le broker en cas de perte de connexion. Grafana détecte ainsi les nœuds hors ligne.
- Surveillance d'état : Node-RED vérifie la réception régulière de chaque nœud. Si un topic est silencieux depuis plus de 15 secondes, une alerte 'Node inactif' est générée.

5.4 Gestion des données

InfluxDB stocke toutes les métriques reçues dans un bucket dédié (iot_servers). Chaque mesure est tagguée par server_id et le timestamp est fourni par le nœud IoT. Les politiques de rétention configurées sont :

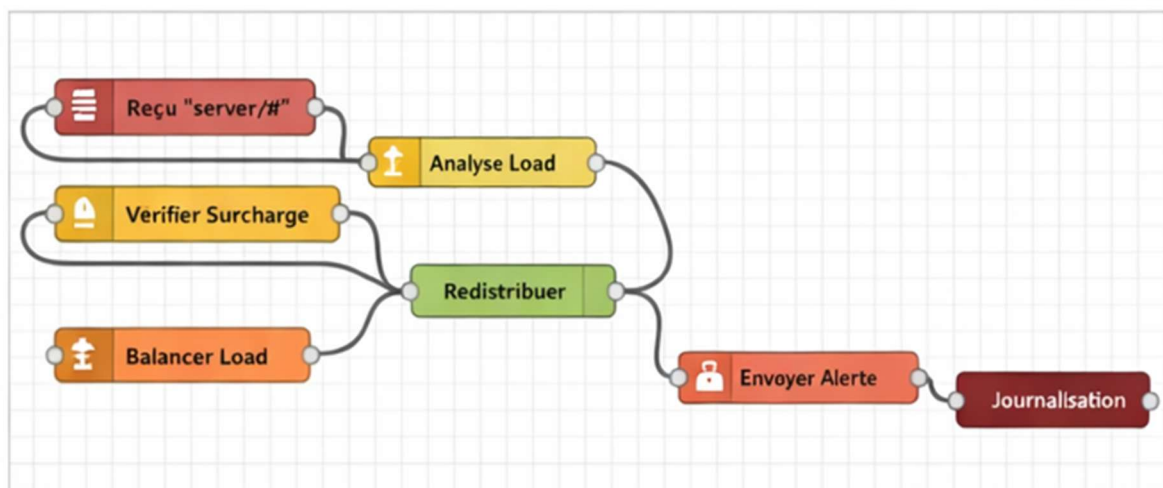
- Données brutes : rétention de 30 jours
- Agrégats horaires (moyenne, max, min) : rétention de 1 an
- Alertes et événements de load balancing : rétention illimitée
-

5.5 Traitement et moteur de règles (Node-RED)

5.5.1 Flux principal de traitement

Node-RED implémente le flux de traitement suivant pour chaque message reçu :

Flux de Contrôle Node-RED



6. Réception du message MQTT depuis Mosquitto
7. Parsing du payload JSON et extraction de server_id, cpu, ram, network, load_score
8. Mise à jour de la table d'état globale des 4 serveurs en mémoire
9. Évaluation des règles de seuil sur le serveur émetteur
10. Si surcharge détectée : identification du serveur cible le moins chargé
11. Publication de la commande de migration sur loadbalancer/commands
12. Publication de l'alerte sur loadbalancer/alerts
13. Écriture de la métrique dans InfluxDB via son API http

5.5.2 Algorithme de load balancing

L'algorithme de redistribution de charge est basé sur la règle du minimum : le serveur cible est celui présentant le load_score le plus faible, à condition que ce score soit inférieur à 70 %. Si aucun serveur ne remplit cette condition, l'alerte est publiée mais aucune migration n'est forcée.

Un mécanisme anti-oscillation (cooldown de 30 secondes) empêche les migrations successives trop rapides sur un même serveur.

5.6 Analytics et détection d'anomalies

- Seuils statiques : CPU > 80 %, RAM > 85 %, Network > 900 Mbps → alerte immédiate
- Variation rapide : augmentation > 20 % du load_score en moins de 30 secondes → alerte précoce
- Tendance croissante : moyenne glissante sur 5 minutes supérieure à 70 % → alerte de tendance
- Serveur inactif : absence de publication depuis 15 secondes → alerte LWT

5.7 Interface utilisateur — Grafana

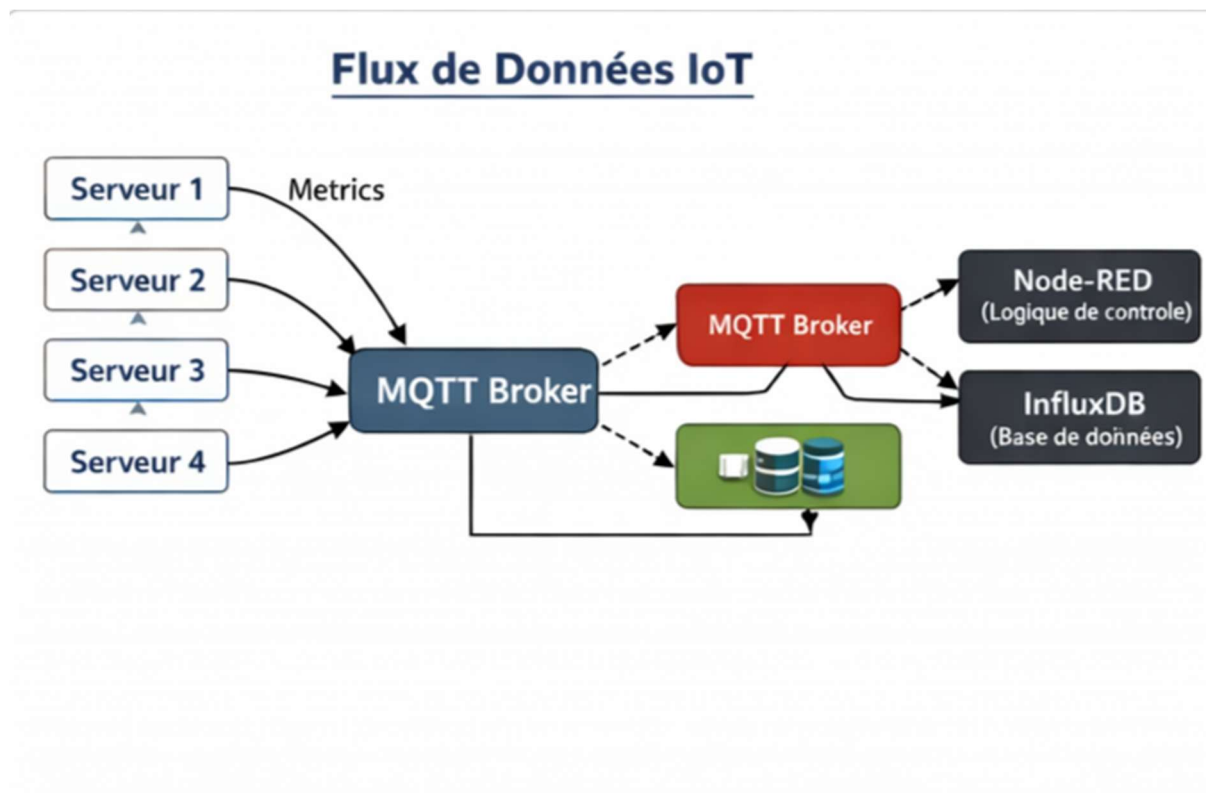
Le tableau de bord Grafana est organisé en 4 panneaux principaux :

- Vue globale : 4 jauges côte à côte affichant le load_score instantané de chaque serveur (code couleur : vert < 60 %, orange 60-80 %, rouge > 80 %)
- Graphiques historiques : courbes de CPU, RAM et réseau sur les 30 dernières minutes, une courbe par serveur
- Journal des événements : table chronologique des alertes et commandes de load balancing
- Panneau d'état : indicateur en ligne / hors ligne de chaque nœud IoT

Les alertes Grafana sont configurées pour envoyer une notification par email ou webhook lorsqu'un seuil est dépassé depuis plus de 30 secondes.

5.8 Schéma fonctionnel du flux de données

Le flux de données complet, de bout en bout, est le suivant :



14. Scripts Python (Servers 1-4) génèrent métriques → publient sur MQTT topic `servers/{id}/{metric}`
15. Mosquitto reçoit et route les messages vers les abonnés
16. Node-RED reçoit → évalue → écrit dans InfluxDB et publie commandes/alertes si nécessaire
17. InfluxDB stocke toutes les séries temporelles
18. Grafana interroge InfluxDB et affiche le tableau de bord en temps réel
19. Nœuds IoT souscrivent à `loadbalancer/commands` et simulent la réduction de charge

6. Aspects Sécurité

Même dans un déploiement local, les aspects de sécurité sont pris en compte :

- Authentification MQTT : chaque nœud IoT s'authentifie avec un identifiant unique et un mot de passe
- Contrôle d'accès : les ACL Mosquitto limitent chaque nœud à ses propres topics de publication
- API InfluxDB : protégée par token d'accès, avec des permissions en écriture seule pour Node-RED
- Grafana : accès protégé par authentification utilisateur/mot de passe
- Isolation réseau : tous les services écoutent uniquement sur 127.0.0.1 en environnement de test
- TLS (prévision) : pour un déploiement en production, les communications MQTT et HTTP seraient chiffrées avec TLS

7. Conclusion

Ce rapport présente la conception complète d'un système IoT de monitoring et de load balancing intelligent de 4 serveurs. Le système répond à l'ensemble des exigences du cahier des charges :

- Acquisition de données via des nœuds IoT simulés en Python
- Transmission via le protocole MQTT et le broker Mosquitto
- Détection de surcharge et redistribution automatique de charge par Node-RED
- Stockage des séries temporelles dans InfluxDB
- Visualisation en temps réel et alertes dans Grafana

L'architecture choisie — plateforme IoT personnalisée locale — garantit une faible latence, une autonomie totale sans Internet, et une totale maîtrise des données. La conception est modulaire et extensible : l'ajout de nouveaux serveurs ne nécessite que la création d'un nouveau script Python et la mise à jour des topics MQTT.

La partie prototypage (Phase II) consistera à implémenter l'ensemble des composants décrits dans ce rapport, à réaliser des tests fonctionnels complets et à produire une démonstration vidéo du système en fonctionnement.

8. Références

- MQTT.org — Spécification du protocole MQTT v5.0 — <https://mqtt.org>
- Eclipse Mosquitto — Documentation officielle du broker — <https://mosquitto.org>
- Node-RED — Documentation et flows de référence — <https://nodered.org>
- InfluxData — Documentation InfluxDB 2.x — <https://docs.influxdata.com>
- Grafana Labs — Documentation Grafana — <https://grafana.com/docs>
- Eclipse Paho — Client MQTT Python — <https://pypi.org/project/paho-mqtt>
- ETSI EN 303 645 — Cybersecurity for Consumer IoT — 2020